

Hough Circle Detection

Hough Circle Detection

- Preface
- Example Results
- Principle Explanation
 - Hough Circle Transform
 - Parameter Definitions
- Code Overview
 - Initialize Camera and Display Module
 - Downsample and Preprocess
 - Execute Hough Circle Detection
 - Map Coordinates Back and Draw
 - Resource Release
 - Parameter Adjustment Notes

Preface

This tutorial is applicable to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. Download from Jianan official website https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/

选择开发板

CanMV K230 V1.0/V1.1

CanMV K230 V3.0

庐山派 K230

庐山派 Lite K230D

Yahboom K230

01Studio K230

01Studio K230 Emmc

Hiwonder K230

WonderMK K230

GT6700 K230

勘智 KENDRYTE

Yahboom K230

MicroPython Build for Yahboom K230

当前下载镜像:
[CanMV_K230_YAHBOOM_micropython_v1.8-20-g9b151af_nncase_v2.11.0.img.gz](#)

最新每日构建版本

Daily-0728

编译日期: 2026-07-28

MD5: 66f0622fc6b8d22e55f552a40458d2fa

立即下载

极简 USB 烧录指南 (推荐)

⚠ 镜像仅可烧录到对应的开发板, 请务必选择与当前板型完全匹配的镜像, 不能混烧其他板型的镜像。否则可能导致硬件损坏!!!

1 准备工具

下载并打开 [K230BurningTool](#) 烧录工具, 并加载固件。

2 进入烧录模式 (根据硬件情况选择)

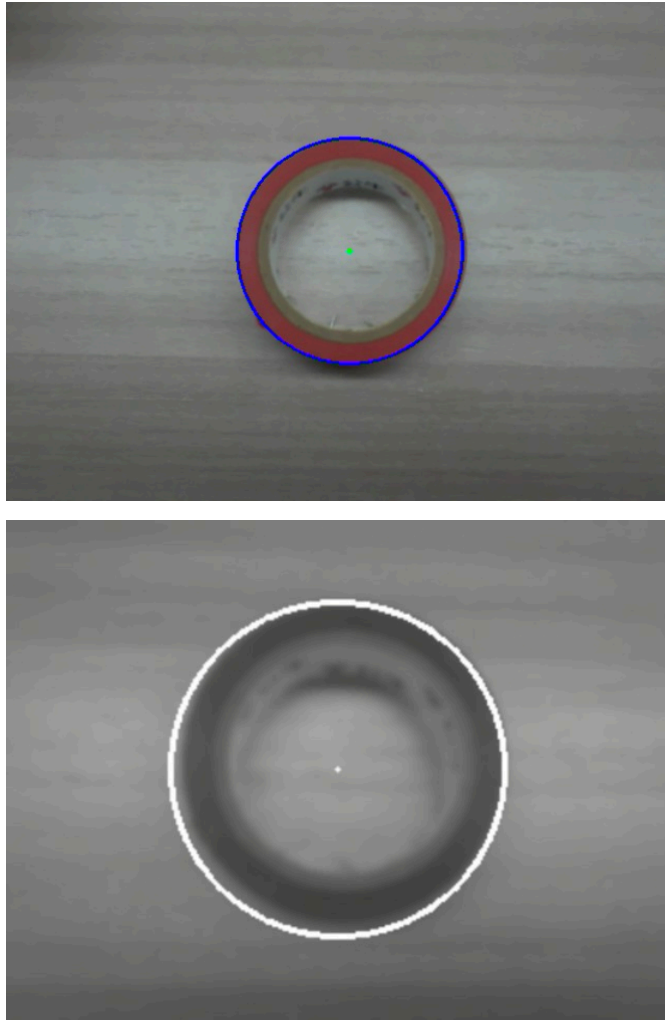
有 BOOT 键的板子

无 BOOT 键的板子

Example Results

Run the example code for this section [05.camera_hough_circles.py](#).

This section downsamples the 640x480 camera feed to 320x240, applies Gaussian blur and Hough circle detection on the grayscale image, then scales the detected circle center and radius coordinates by 2 and draws them back onto the original image.



Principle Explanation

Hough Circle Transform

A circle can be represented by its center (x, y) and radius r . The Hough circle transform votes for possible centers and radii in parameter space based on edge directions in the image; candidates with higher vote counts are returned as circles.

Hough circle computation is intensive, and it can easily combine table edges, reflections, cables, and scattered arcs into false circles. Therefore, this example first downsamples and applies Gaussian blur to reduce pixel count and noisy edges.

Parameter Definitions

```
circles = cv2.HoughCircles(  
    blur, 3,  
    dp=1,  
    minDist=60,  
    param1=100,  
    param2=70,  
    minRadius=10,  
    maxRadius=100  
)
```

Parameter	Current Value	Description
method	3	The circle detection method value accepted by current K230 binding
dp	1	Accumulator resolution matches input image
minDist	60	Minimum distance between two circle centers
param1	100	Upper threshold for internal edge detection
param2	70	Circle center accumulator threshold, higher is stricter
minRadius	10	Minimum radius on 320x240 image
maxRadius	100	Maximum radius on 320x240 image

Code Overview

Initialize Camera and Display Module

```

W, H = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_id=True)
sensor.run()
time.sleep(0.5)

```

Downsample and Preprocess

```

small = cv2.resize(img_np, (320, 240))
gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (7, 7), 1.5)

```

After downsampling, pixel count is about one quarter of the original. The 7x7 Gaussian kernel and sigma=1.5 are used to suppress fragmented edges, but may also weaken very small or blurry circles.

Execute Hough Circle Detection

```

circles = cv2.HoughCircles(
    blur, 3,
    dp=1, minDist=60,
    param1=100, param2=70,
    minRadius=10, maxRadius=100
)

```

When the return value is None, no circles meeting the conditions were found. The current example reads the first three values as center and radius according to the array layout returned by K230 binding.

Map Coordinates Back and Draw

```
if circles is not None:
    for i in range(circles.shape[0]):
        if i >= 30:
            break
        x = int(circles[i, 0]) * 2
        y = int(circles[i, 1]) * 2
        r = int(circles[i, 2]) * 2
        cv2.circle(img_np, (x, y), 3, (0, 255, 0), -1)
        cv2.circle(img_np, (x, y), r, (0, 0, 255), 2)
```

Detection is performed at half resolution, so both center and radius are multiplied by 2. The source code draws at most 30 circles to prevent excessive false circles from slowing down display.

Resource Release

```
finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Restrict Radius First:** Tightening minRadius and maxRadius according to actual target size is usually more effective than simply raising thresholds.
- **Too Many False Detections:** Increase param2, minDist, and param1 in order.
- **Too Many Missed Detections:** Appropriately decrease param2, reduce Gaussian kernel size, confirm radius range covers the target.
- **Coordinate Scale:** All half-resolution parameters should be understood for 320x240; multiply by 2 when drawing back to 640x480.
- **Complex Background:** High-reliability applications should add ROI, color masks, contour circularity or circle edge support rate verification; cannot rely solely on a single HoughCircles() call.
- **Performance:** This algorithm performs global search, so frame rate is usually lower than simple filtering. Disabling IDE image feedback can help distinguish between computation and transmission overhead.